

Guide du développeur · PDF Equilibrist

[<-](#) [Index](#) · [Architecture](#) · [Operations](#) · [Build](#)

■ Prérequis

- Python 3.12
- Conda (Miniconda ou Anaconda)
- Windows 10/11 (l'app est Windows-only pour l'instant)

■ Mise en place de l'environnement

```
# Cloner le projet
git clone <url> PDF-Equilibrist
cd PDF-Equilibrist

# Restaurer l'environnement conda
conda env create -p D:\Developpement\CONDA\envs\Equilibrist -f environment.yml

# Activer
conda activate D:\Developpement\CONDA\envs\Equilibrist

# Installer le projet en mode éditable
pip install -e .
```

■ Lancer l'application

```
python src/pdf_equilibrist/main.py
```

Le splash screen apparaît, puis la fenêtre principale.

En mode dev, `resource_path()` résout les assets depuis la racine du projet.

■ Structure d'un onglet ribbon

Chaque onglet du ribbon est un `QWidget` dans `ui/tabs/`. Pour en créer un :

```
# ui/tabs/tab_exemple.py
"""
tabs/tab_exemple.py · Onglet "Exemple"
Description courte du rôle de l'onglet.
"""

from PyQt6.QtWidgets import QWidget, QHBoxLayout
from pdf_equilibrist.ui.widgets import RibbonButton, RibbonGroup
from pdf_equilibrist.core.document import Document

class TabExemple(QWidget):
    def __init__(self, document: Document, viewer=None):
        super().__init__()
        self.document = document  # référence rebindée par MainWindow._rebind_doc()
        self.viewer = viewer
```

Puis dans `main_window.py` :

```
from pdf_equilibrist.ui.tabs.tab_exemple import TabExemple

self._tab_exemple = TabExemple(self._current_doc, self.viewer)
# Ajouter au ribbon_tabbar et ribbon_stack
# Ajouter à la liste dans _rebind_doc()
```

■ Ajouter une opération PDF

Les opérations dans `operations/` sont des fonctions pures (sans UI) :

```
# operations/edit.py (exemple d'ajout)
def add_border(doc: fitz.Document, page_index: int, color: tuple = (1, 0, 0)):
    """
    Ajoute un cadre rouge sur une page.

    Parameters
    -----
    doc : fitz.Document
        Document à modifier en place.
    page_index : int
        Numéro de page 0-based.
    color : tuple
        Couleur RGB normalisée 0.1.
    """
    page = doc[page_index]
    page.draw_rect(page.rect, color=color, width=2)
```

Règles :

- Pas d'import PyQt6 dans `operations/`
- Retourner des types standards Python (`Path`, `list`, `bool`·)
- Documenter les paramètres et les cas limites
- Le tab appelant gère l'UI (dialogues, progression) et émet `document.changed`

■ Convention de nommage

Élément	Convention	Exemple
Fichiers	<code>`snake_case`</code>	<code>`tab_modifier.py`</code>
Classes	<code>`PascalCase`</code>	<code>`FloatingItem`</code>
Méthodes publiques	<code>`snake_case`</code>	<code>`enter_edit_mode()`</code>
Méthodes privées	<code>`_snake_case`</code>	<code>`_on_doc_changed()`</code>
Signaux Qt	<code>`snake_case`</code>	<code>`zoom_changed`</code>
Constantes	<code>`UPPER_SNAKE`</code>	<code>`ACCENT`, `THUMB_H`</code>

■ Règles de contribution

1. Signaux Qt

Toujours émettre `document.changed` après une modification du `fitz.Document`.

Ne jamais appeler `viewer.refresh()` ou `thumbs.rebuild()` directement depuis un tab.

2. Chemins d'assets

Utiliser `resource_path()` de `utils.py` pour tout accès à un fichier asset.

Ne jamais utiliser de chemins relatifs durs.

3. Operations sans UI

Les fonctions de `operations/` ne doivent pas importer `PyQt6`.

Si une opération nécessite un retour d'information à l'UI, le faire via la valeur de retour (pas via des dialogues internes).

4. Gestion des erreurs

Envelopper les opérations potentiellement échouantes dans `try/except` dans le `tab` (pas dans l'opération), et afficher l'erreur via `show_error()`.

5. Rebind multi-documents

Si un `tab` stocke un état interne lié au document (ex. `_edit_blocks` dans `tab_modifier`), le nettoyer dans une méthode déclenchée par `document.changed` ou lors du `rebind`.

■ Ajouter un dialogue réutilisable

Dans `ui/dialogs.py` :

```
def ask_my_param(parent, title: str) -> str | None:
    """Retourne la valeur saisie ou None si annulé."""
    dlg = QDialog(parent)
    dlg.setWindowTitle(title)
    dlg.setStyleSheet(DIALOG_STYLE)
    layout = QVBoxLayout(dlg)
    # ... construire le dialogue
    _ok_cancel(dlg, layout)  # ajoute les boutons OK/Annuler
    return result if dlg.exec() == QDialog.DialogCode.Accepted else None
```

■ Lancer les tests

```
# Lancer tous les tests
pytest tests/

# Un test spécifique
pytest tests/test_operations/test_edit.py::test_extract_text_blocks -v
```

Les tests de `operations/` peuvent s'exécuter sans interface graphique.

Les tests d'UI nécessitent un `display` (ou `QApplication` avec `offscreen platform`).

■ Mettre à jour l'environnement

Après installation de nouvelles dépendances :

```
# Régénérer le fichier de freeze
pip freeze > requirements_freeze.txt

# Mettre à jour environment.yml manuellement ou via :
conda env export -p D:\Developpement\CONDA\envs\Equilibrist --no-builds > environment.yml
```